Présenter le projet avec précision

Pitch de 45 secondes
LostChain est une plateforme d'objets perdus et retrouvés pour un campus. Un membre déclare un objet, un autre soumet une demande de récupération, puis le déclarant décide. Une restitution n'est terminée qu'après confirmation des deux participants. CampusIdentity gère les inscriptions par wallet ; LostItemRegistry applique les règles métier. Les contrats sont déployés sur Sepolia ; les photos et les preuves sont conservées hors chaîne.

Ce que la grille attend

Critère	Points	Ce que je dois démontrer
Concepts et conception	4	Justifier le besoin, la blockchain et l'architecture.
Contrats, sécurité et tests	6	Lire le code, expliquer les protections et leurs limites.
Questions techniques	7	Répondre avec autonomie, exemples et preuves.
Utilisation de l'IA	3	Expliquer comment j'ai vérifié et corrigé les résultats.

Pourquoi une blockchain ?

Elle rend les transitions et confirmations vérifiables dans un registre partagé, sans décision d'un administrateur applicatif. La valeur visée est la traçabilité des engagements entre participants. Une base classique serait plus simple, moins coûteuse et plus adaptée à la confidentialité : la blockchain est ici un choix de traçabilité, pas une obligation technique.

Que ne prouve pas le contrat ?

Il prouve qu'une adresse a envoyé une transaction et que les conditions codées étaient réunies. Il ne prouve pas l'identité civile, l'appartenance au campus, la propriété physique ou la réalité de la remise. Deux personnes peuvent confirmer une remise fictive.

État à annoncer honnêtement

Deux contrats déployés sur Sepolia ; inscription et interaction avec le registre testées manuellement sur ce réseau. Parcours complet à deux wallets validé localement, **pas entièrement sur Sepolia**. Frontend, API et photos encore sur l'ordinateur. Pas de mesure de couverture ni d'audit externe à présenter comme acquis.

Expliquer l’architecture

Lecture du système
React / React Router → contexte applicatif → adaptateur de données.
Prototype : localStorage uniquement.
Blockchain : Ethers.js + MetaMask → contrats ; appels HTTP → API Node.js → SQLite et RPC.
Réseaux : Hardhat 31337 ou Ethereum Sepolia 11155111.

Composant	Responsabilité
React + Vite	Pages, formulaires, navigation et état affiché. Les boutons ne sont pas une barrière de sécurité.
MetaMask	Conserve les clés et demande la signature des messages et transactions.
Ethers.js	Encode les appels via l’ABI, utilise le signer et attend les reçus.
CampusIdentity	Inscription unique par wallet, rôle et nom public.
LostItemRegistry	Déclarations, demandes, décisions et double confirmation.
API Node + SQLite	Emails, photos, preuves privées, rendez-vous, sessions et lecture des notifications.
RPC / index d’événements	Accès à la chaîne ; reconstruction des activités et notifications.

Pourquoi deux contrats ?

Séparer l’identité des règles de récupération. Le registre reçoit l’adresse de CampusIdentity dans son constructeur et appelle isRegistered(msg.sender). Les responsabilités sont plus lisibles et testables ; les interfaces doivent rester compatibles.

Le système est-il totalement décentralisé ?

Non. Les états métier sont sur la blockchain, mais l’interface, l’API et les pièces jointes dépendent encore d’un serveur. Si l’API tombe, les contrats existent toujours ; les preuves/photos et certaines vues applicatives deviennent indisponibles.

Que stocke-t-on où ?

Sur chaîne : nom, rôle, wallets, titre, catégorie, description, localisation, états et confirmations. Hors chaîne : email, photos, rendez-vous et preuve détaillée. Le champ message de la demande reçoit, via le frontend, un engagement hashé et salé de la preuve. Éviter toute donnée sensible dans les champs publics.

Maîtriser CampusIdentity

Élément	Explication à donner
mapping(address => UserProfile)	Associe le profil à l'adresse du wallet. Une adresse n'est pas une identité civile.
Role	Student = 0 ; Staff = 1. Les deux ont les mêmes permissions.
registerUser(name, role)	Refuse une inscription déjà existante et un nom vide ; écrit le profil et émet UserRegistered.
getUserProfile / isRegistered	Fonctions de lecture du profil ou de son statut.
updateUserProfile(newName)	Le membre modifie son propre nom ; le nouveau nom ne doit pas être vide. Le rôle n'est pas modifiable par cette fonction.
getUserCount / getRegisteredUsers	Décompte et liste des adresses inscrites.

Comment le frontend authentifie-t-il un wallet ?

L'API fournit un challenge à usage unique avec origine, adresse, chaîne, déploiement, nonce et expiration. MetaMask signe le message. L'API vérifie la signature, consomme le challenge et crée une session avec cookie HttpOnly / SameSite. Le challenge expire après 5 minutes ; la session après 24 heures.

Quelle différence entre signer et s'inscrire ?

Le message de connexion établit le contrôle du wallet pour la session API. L'inscription modifie CampusIdentity : c'est une transaction on-chain, avec gas et reçu. Une signature de connexion n'inscrit pas automatiquement le membre dans le contrat.

Un attaquant peut-il choisir Staff ?

Oui, le rôle est librement choisi dans cette démonstration. Comme les permissions sont identiques, Staff ne donne pas de pouvoir supplémentaire. Cela ne valide aucune affiliation réelle ; un mécanisme de vérification campus serait un travail futur.

Pièges à éviter
« Registered » signifie inscrit dans le contrat, pas étudiant vérifié.
Une inscription unique par adresse n'empêche pas une personne de créer plusieurs wallets.
Un mapping private n'est pas une garantie de confidentialité sur une blockchain publique.
Les contrats actuels n'ont ni rôle Admin ni propriétaire technique configurable.

Lecture à pratiquer

Montrer les deux require de registerUser, l'utilisation de msg.sender, l'écriture du struct et l'émission de UserRegistered. Expliquer pourquoi getUserProfile refuse un wallet non inscrit et pourquoi une validation frontend seule ne suffit pas.

Maîtriser le parcours de récupération

Invariant principal
Completed / Returned n'est atteint que si la demande est Accepted et si reporterConfirmed **ET** claimantConfirmed valent true. Le contrat enregistre séparément les confirmations. L'ordre de confirmation n'a pas d'importance.

Action	Autorisation et transition
Déclarer Lost / Found	Membre inscrit ; titre non vide ; nouvel objet Open.
submitClaim	Inscrit, objet Found et disponible, ni son propre objet ni doublon. Demande Pending ; objet ClaimPending.
acceptClaim	Déclarant uniquement, demande Pending. Demande Accepted ; autres demandes Pending rejetées ; objet ClaimAccepted.
rejectClaim	Déclarant uniquement ; Pending → Rejected. Retour à Open si plus aucune demande en attente.
cancelClaim	Demandeur uniquement ; Pending → Cancelled. Le marqueur hasClaimed reste actif : pas de nouvelle demande pour ce couple wallet/objet.
confirmHandover	Demande Accepted ; déclarant ou demandeur, chacun une fois. Les deux confirmations donnent Completed et Returned.
closeItem	Déclarant uniquement ; objet Open ou ClaimPending. Les demandes Pending sont annulées ; objet Closed.

Comment le contrat connaît-il l'auteur réel ?

msg.sender vient de l'exécution de la transaction signée. On compare cette adresse au reporter stocké ou au claimant. Le navigateur ne choisit pas librement la valeur de msg.sender.

À quoi servent modifiers, require et revert ?

onlyRegistered vérifie l'inscription ; itemExists et claimExists vérifient l'existence. require impose une condition ; revert annule l'exécution et ses modifications. Une transaction échouée après inclusion peut tout de même consommer du gas.

Lost, Found, Open, Returned : quelle différence ?

Lost / Found est le type ; Open, ClaimPending, ClaimAccepted, Returned et Closed sont les états du contrat. L'interface peut afficher Lost / Found sur un objet actif même quand le contrat indique une demande en cours. Returned signifie la double confirmation ; Closed est une fermeture par le déclarant.

Discuter sécurité et limites

Risque	Protection actuelle / limite à expliquer
Action d'un tiers	Comparaisons reporter/claimant avec msg.sender, vérifiées dans le contrat ; tests de refus.
Double demande / double confirmation	hasClaimed interdit les doublons ; booléens et statut Accepted encadrent les confirmations.
Preuve modifiée	L'API compare le hash salé de la preuve à l'engagement on-chain. Cela contrôle l'intégrité, pas la véracité de la preuve.
Rejeu du message de connexion	Challenge expirant et consommé, origine/chaîne/déploiement inclus, signature et adresse vérifiées.
Accès aux preuves	API limitée au déclarant et au demandeur. La base n'est pas chiffrée par ce mécanisme ; l'opérateur reste une partie de confiance.
Abus de demandes / coût en gas	acceptClaim et closeItem parcourent les demandes : coût croissant, risque de blocage à grande échelle. Limites, traitement par lots ou invalidation logique à étudier.
Multiplication de comptes	Une personne peut créer plusieurs wallets ; aucune protection Sybil ou vérification campus complète.
Validation contournée	Les limites frontend sur les textes/photos ne sont pas toutes reproduites dans le contrat. Un appel direct peut les contourner.

Le contrat est-il protégé contre toute réentrance ?

Je ne peux pas affirmer une sécurité absolue. Le code actuel ne transfère pas d'ETH et ne rappelle pas le claimant/reporter ; le contrôle d'inscription est une lecture du contrat référencé. Ce chemin réduit la surface classique de réentrance, mais ne remplace pas une analyse de sécurité.

Quelles limites fonctionnelles importantes ?

Pas de litige, de médiation, de délai ni de récupération si un participant ne confirme jamais. Le nom reste public malgré le choix d'affichage anonyme. Une collusion est possible. Les getters qui renvoient toutes les données et l'index en mémoire devront évoluer pour de gros volumes.

Améliorations proposées, non réalisées
Tests de charge en gas ; limitation ou refonte des boucles ; pagination ; index persistant ; politique de confidentialité ; vérification d'email ; procédure de litige. Modifier un contrat déployé implique une stratégie de nouveau déploiement/migration : aucun proxy upgradeable n'est prévu ici.

Présenter des preuves de validation

Dernière vérification enregistrée : 34 tests frontend/services, 26 tests contrats et 5 tests d'intégration réussis ; vérification TypeScript et builds réussis. Ces nombres sont un état du projet, pas un pourcentage de couverture.

Commande	Ce que je montre
npm test	Parcours prototype, persistance, notifications, données de preuve et configuration réseau.
npm run chain:typecheck	Compilation et vérification des types TypeScript.
npm run chain:test	CampusIdentity et LostItemRegistry sur environnement de test Hardhat.
npm run test:integration	Repository + API + contrats ; nécessite le nœud local. Contrats temporaires et retour au snapshot.
npm run build:sepolia	Construction du frontend Sepolia ; ne teste pas toutes les interactions publiques.

Trois scénarios de sécurité à expliquer sans hésitation

1. Un tiers tente d'accepter : le require sur le reporter échoue ; la demande reste Pending.
2. Une seule confirmation : son booléen devient true, la demande reste Accepted ; Returned n'est pas encore atteint.
3. Demande sur Lost ou demande doublonnée : l'appel est refusé avant la création de la nouvelle demande.

Ce qui est vérifié, ce qui ne l'est pas

Local : parcours complet à deux comptes, deux ordres de confirmation dans les tests, erreurs d'accès, refus/annulation/fermeture et persistance.
Sepolia : deux contrats déployés ; inscription et interaction métier testées manuellement. La recette complète à deux wallets a été volontairement arrêtée et reste non validée.
Non fourni : rapport de couverture, audit externe, campagne complète de tests de vulnérabilités ou validation sous forte charge.

Plan de démonstration en 3 minutes

1. Montrer les adresses des contrats et une transaction réussie sur Sepolia.
2. Montrer l'inscription et une déclaration dans l'application.
3. Expliquer le parcours de restitution avec les tests locaux comme preuve des deux confirmations.
4. Ouvrir confirmHandover dans le code et identifier les contrôles.
5. Terminer sur une limite réelle et une amélioration proposée.

Ne pas lancer les tests à snapshot pendant que quelqu'un envoie des transactions sur le même nœud local. Ne pas redéployer des contrats juste pour la présentation. Préparer des captures de secours sans secrets.

Réponses courtes aux questions du jury

Hardhat et Sepolia : quelle différence ?

Hardhat fournit un environnement de développement local contrôlé, rapide et réinitialisable. Sepolia est un réseau de test public : les transactions prennent du temps et nécessitent des ETH de test. Le chain ID évite de confondre les réseaux : 31337 contre 11155111.

Qu’est-ce qu’une ABI ?

La description des fonctions, événements et types du contrat. Ethers.js l’utilise pour encoder les appels et décoder les résultats. L’ABI n’est ni la clé privée ni l’adresse du contrat ; une ABI correcte ne prouve pas à elle seule que l’adresse vise le bon code.

Qu’est-ce qu’un événement ?

Une trace émise pendant une transaction, par exemple ClaimAccepted ou ItemReturned. L’API lit les logs pour créer activités et notifications. L’événement facilite l’observation ; les variables d’état restent la référence des règles métier.

Lecture et écriture ont-elles le même coût ?

Un eth_call de lecture ne crée pas de transaction payante pour l’utilisateur. Une écriture doit être signée et incluse dans un bloc, avec du gas. Les requêtes RPC ont néanmoins des limites de disponibilité et de quota.

Que se passe-t-il si la photo échoue après la transaction ?

La blockchain et SQLite ne forment pas une transaction atomique. Le repository réserve le stockage navigateur, puis conserve les métadonnées dans une file par wallet/déploiement. Retry data sync renvoie les métadonnées sans recréer l’objet ou la demande on-chain.

Un hash garantit-il la confidentialité ?

Un hash n’est pas un chiffrement. Le sel aléatoire complique la vérification de suppositions sur le contenu ; la preuve originale reste dans la base. Sans sa sauvegarde, le hash ne permet pas de reconstruire la photo ou le texte.

Pourquoi le site est-il encore local si les contrats sont publics ?

Le déploiement des contrats et l’hébergement web sont deux choses différentes. Sepolia conserve les contrats ; le frontend, l’API et SQLite tournent encore sur l’ordinateur. Un accès public partagé exige hébergement, HTTPS, proxy API, stockage et sauvegardes.

Comment répondre à une question inconnue ?

Nommer ce que je sais, ouvrir la fonction ou le test pertinent, expliquer ce qu’il faudrait vérifier. Ne pas inventer une protection, une fonctionnalité ou une preuve de test absente.

Expliquer l'IA et préparer sa prise de parole

Formulation à adapter à tes contributions réelles
Nous avons utilisé l'IA pour aider à structurer le frontend, préparer l'intégration et proposer des tests et configurations. Les contrats initiaux ont été créés manuellement selon notre travail déclaré. Nous avons confronté les propositions au code, exécuté des tests et réalisé des vérifications manuelles. Je dois pouvoir expliquer chaque partie que je présente, y compris celle produite avec assistance.

Exemples concrets de vérification dans ce projet

Situation	Ce qu'elle montre
Port déjà occupé	Le démarrage initial pouvait créer des doublons ; la détection des services et leur réutilisation ont été ajoutées et testées.
Tests sur comptes déjà inscrits	Les fixtures supposaient des comptes vierges. Elles déploient maintenant leurs propres contrats dans un snapshot.
Différence contrat / photos	Une confirmation on-chain ne prouve pas la sauvegarde des pièces jointes ; une reprise séparée a été ajoutée.
Frais de déploiement Sepolia	L'alerte de fonds insuffisants a été examinée ; le second contrat a ensuite été confirmé par l'utilisatrice avec un niveau de frais adapté.

Trace d'IA à préparer

Choisir 2 ou 3 échanges réels : demande initiale, proposition obtenue, contrôle effectué, défaut constaté et correction. Conserver les résultats de tests associés. Ne pas inventer un journal de prompts ou revendiquer un audit automatique non réalisé.

Répartition dans le binôme

Chacun décrit sa contribution réelle et connaît les interfaces partagées. Si ton binôme présente CampusIdentity, tu dois néanmoins expliquer comment LostItemRegistry s'y relie. Répéter une explication croisée de registerUser, submitClaim et confirmHandover.

Auto-évaluation express

Sans lire, puis code à l'appui :
1. Expliquer l'architecture en 60 secondes.
2. Montrer où un tiers est refusé.
3. Justifier le ET entre les deux confirmations.
4. Citer trois limites et proposer une amélioration concrète pour chacune.
5. Distinguer ce qui a été testé localement de ce qui a été testé sur Sepolia.

À ne pas dire : « totalement décentralisé », « identité campus certifiée », « anonymat garanti », « tous les risques sont couverts » ou « 100 % de couverture ». Ces affirmations ne sont pas établies par le projet.

Mémo de dernière minute

Adresses Sepolia à conserver

CampusIdentity

`0xC9C143a2Ad6CFB72e41da14c5Cd20CdC11f42dE7`

Ouvrir le contrat sur Sepolia Etherscan

LostItemRegistry

`0xD2192DaCBc7aC9449A0Aaedf4207F478ce78149E`

Ouvrir le contrat sur Sepolia Etherscan

Démarrage pour la démonstration

```
npm run sepolia:start
```

Frontend Sepolia : http://127.0.0.1:5175 - API locale : port 3003.

```
npm run local:start
```

Frontend Hardhat : http://127.0.0.1:5174 - RPC : port 8545.

```
npm run dev
```

Prototype sans blockchain : port 5173, données navigateur.

Fichiers à savoir retrouver

Fichier ou dossier	Pourquoi l'ouvrir
blockchain/contracts/CampusIdentity.sol	Inscription, rôles et profil.
blockchain/contracts/LostItemRegistry.sol	Permissions, états et double confirmation.
blockchain/test/	Preuves des comportements et refus dans les contrats.
src/services/chain/repository.js	Signature, transactions, preuve salée et reprise des métadonnées.
server/chain-app.js	Session wallet, accès aux preuves et métadonnées.
tests/chain.integration.js	Validation de bout en bout locale avec contrats isolés.
server/event-index.js	Lecture incrémentale des événements ; index en mémoire.
README.md / SEPOLIA_SETUP.md	Installation, configuration, limites et commandes.

Checklist avant de commencer

MetaMask déverrouillé sur le bon compte et réseau ; services locaux actifs ; solde de test suffisant ; onglet Etherscan prêt ; résultats de tests disponibles ; aucune clé privée/API dans les captures ; rôle de chaque membre du binôme défini.

Base : grille fournie, code du dépôt et validations enregistrées au 10 novembre 2026. Reformuler ces réponses et les vérifier dans le code. Hébergement public, audit, couverture et publication des sources Etherscan ne sont pas attestés par cette fiche.